

Implementing Agent Systems

The design process of Prometheus results in descriptors and diagrams that describe a range of design entities such as goals, functionalities, agents, capabilities, plans, percepts, actions, messages, data, protocols and scenarios. However, not all of these design entities are carried through to implementation. For example, functionalities are used to determine the agent types but they do not correspond to any run-time entity. Roughly speaking, the entities that are implemented are those that are produced in the detailed design phase: namely, agents, capabilities, plans, messages and beliefs; as well as actions, percepts and goals. In this chapter, we briefly look at how we undergo a transition from detailed design to implementation and how certain agent platforms support a very direct mapping of these concepts.

Clearly, the concepts of messages, plans and beliefs (as well as agents and capabilities) do not map directly into object-oriented languages such as C++ or Java. Although it is possible to implement an agent-oriented design using an object-oriented language, this is akin to trying to realize an object-oriented design in a non-OO language such as C: awkward and difficult to maintain.

10.1 AGENT PLATFORMS

Fortunately, there exist a range of ‘agent-oriented’ programming languages. Generally, these languages also provide libraries and infrastructure, such as naming services (‘white pages’), and so we shall use the term ‘agent platform’. When considering which agent platform to use, it is important to be aware that not all agent platforms support plan-based agents. Roughly speaking, there are three classes of agent platforms:

1. Those that focus on internal agent reasoning and support plans, goals, and so on.

Examples: PRS¹ (Georgeff and Lansky 1986; Ingrand *et al.* 1992), UMPRS² (Lee *et al.* 1994), JAM³ (Huber 1999), JACK⁴ (Busetta *et al.* 1999), DECAF⁵, Zeus⁶, AgentBuilder⁷ and JADEX⁸ (which is an extension of JADE⁹).

2. Those that focus on inter-agent communications. These usually provide infrastructure for inter-agent communication (for example, a way of sending messages in a certain syntax) as well as facilities for locating agents on the basis of names (white pages) and/or a description of the service that the agents provide (yellow pages). More recent platforms in this class tend to conform to the FIPA¹⁰ standards.

Examples: JADE¹¹, Zeus¹², OAA¹³ (Cheyer and Martin 2001)

3. Those that focus on mobile agents.

Examples: Grasshopper¹⁴, D'Agents¹⁵, Aglets¹⁶

Although inter-agent communication is a specialty of class 2 platforms, all (realistic) platforms provide some support for agent communication (for example, although JACK is listed as being a class 1 platform, it provides support for agent messaging and for locating agents by name). A few platforms provide support for multiple areas. For example, a FIPA-compliance module for JACK is available¹⁷. Another example is JADEX¹⁸, which extends the FIPA-platform JADE with plans, events, goals and beliefs. Another example is JAM, which is primarily a Belief-Desire-Intention (BDI) architecture (class 1), but provides support for agent mobility.

The first class is most useful in terms of providing support for implementing designs developed by following the full Prometheus methodology (including final detailed design).

The aim of this chapter is not to provide a detailed survey of a wide range of platforms. The range and status of available agent platforms changes and the most up-to-date information can be found online. Thus, we give pointers to existing surveys rather than provide a survey that will be out of date by the time this book is published. Two sites that provide extensive lists of tools are

¹<http://www.ai.sri.com/~prs/>

²http://www.marcush.net/IRS/irs_downloads.html

³http://www.marcush.net/IRS/irs_downloads.html

⁴<http://www.agent-software.com>

⁵<http://www.eecis.udel.edu/~decaf/>

⁶<http://more.btexact.com/projects/agents/zeus/>

⁷<http://www.agentbuilder.com/>

⁸See <http://sourceforge.net/projects/jadex>

⁹<http://sharon.cselit.it/projects/jade/>

¹⁰Foundation for Intelligent Physical Agents, <http://www.fipa.org>

¹¹<http://sharon.cselit.it/projects/jade/>

¹²<http://more.btexact.com/projects/agents/zeus/>

¹³<http://www.ai.sri.com/~oaa/>

¹⁴<http://www.grasshopper.de/>

¹⁵<http://agent.cs.dartmouth.edu/>

¹⁶<http://www.trl.ibm.com/aglets/>

¹⁷From <http://www.cs.rmit.edu.au/agents/protocols/>

¹⁸See <http://sourceforge.net/projects/jadex>

- [http://www.cems.uwe.ac.uk/~rsmith/ECOMAS/agent_toolkit_list_\(courtesy_of_bt\).htm](http://www.cems.uwe.ac.uk/~rsmith/ECOMAS/agent_toolkit_list_(courtesy_of_bt).htm)¹⁹
- <http://www.agentbuilder.com/AgentTools/index.html>

Additionally, Luck *et al.* (2004) discusses a number of platforms.

In the remainder of this chapter, we focus on illustrating how the detailed design that has been produced can be implemented using a plan-based agent system. Specifically, we show the close match between the results of the detailed design and the concepts supported by agent platforms such as JACK²⁰. We begin by briefly describing the JACK platform, then discuss issues in mapping the detailed design entities produced by following Prometheus to JACK and give JACK code corresponding to a small part of the detailed design for the book store example.

10.2 JACK

JACK is an agent platform based on the BDI model. It views an agent as having plans that are triggered by events where messages are viewed as a specific sub-type of event. A JACK program consists of declarations of entities: agents, capabilities, plans, events and beliefsets (also termed ‘databases’). Each declaration links to other entities; for example, an agent declaration specifies what plans and capabilities it contains, what beliefsets it has, what messages it receives and sends and what events it posts internally.

Many of the Prometheus concepts map directly to JACK. For example, a Prometheus agent just becomes a JACK agent. JACK also supports capabilities (Busetta *et al.* 1999) and so capabilities in Prometheus map directly to JACK capabilities.

JACK does not have concepts corresponding to percepts and actions. We represent percepts as events. Actions are simply performed within the plan body using Java code.

Also, BDI systems (including JACK) do not implement goals directly. Instead, they model the acquisition of a new goal as an event. Thus, goals are realized by creating an event type corresponding to the goal. As discussed by Thangarajah *et al.* (2002), this has problems such as top-level goals being dropped if there are no applicable plans in the current situation.

10.3 EXAMPLE

As an example, we focus on the highlighted portion of Figure 10.1²¹, which sits within the Ordering capability that is within the *Stock managing* capability within the Stock Manager agent type (see Figure 10.2). This excerpt shows an (internal) message (*Stock low*) that triggers a plan (*Add to order*). The plan makes use of the *Stock Orders* and *Monthly Order* databases, and posts a number of internal messages that trigger further

¹⁹The text ‘(courtesy_of_bt)’ is part of the URL.

²⁰A commercial product of Agent Oriented Software, <http://www.agent-software.com>

²¹The figure focuses on the *Add to order* plan and its context and is incomplete – the complete capability overview diagram is in Figure 10.5 on page 132.

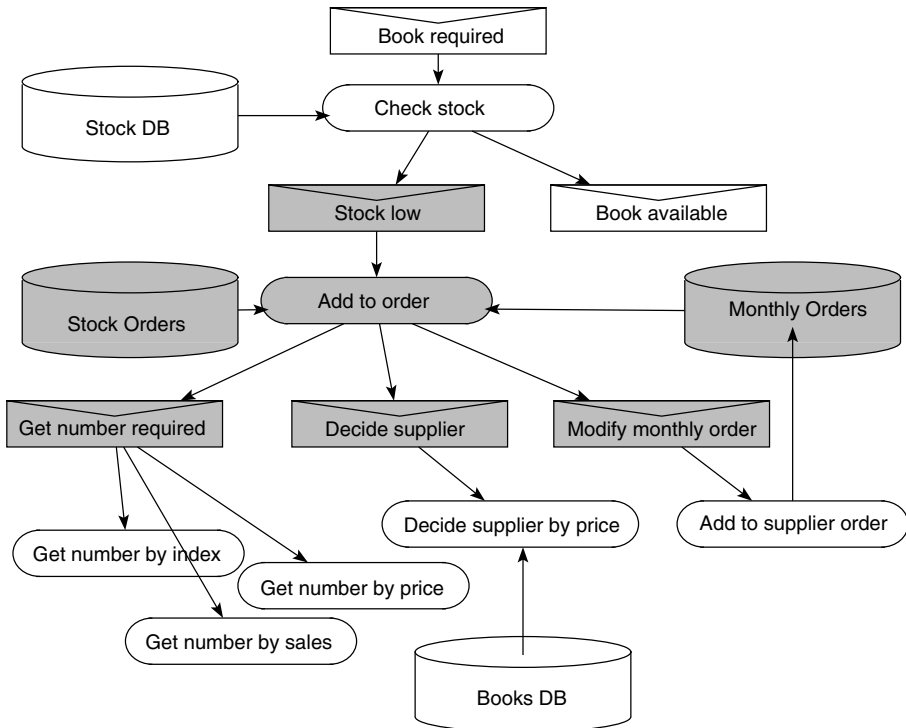


Figure 10.1 Low stock situation within the Ordering capability

plans. The *Stock Orders* database is internal to the agent (but external to the capabilities). The *Monthly Order* database is internal to the *Ordering* capability.

In addition to the database, there is also an object type *book record* that is carried by the messages. This contains information about the book that has low stock, including the book identifier (*bookid*), the *wholesaleprice* and the (optional) *category* of the book.

We consider in turn agents, capabilities, data, messages/events²², and plans. For each entity type, we give an example descriptor, discuss the issues in mapping the design entity to JACK code and give sample JACK code. The code we give is skeleton code and is intended to illustrate how individual design entities map to JACK code, not to be complete and detailed²³.

10.3.1 AGENTS

We begin with declarations for the agent and two capabilities shown in Figure 10.2. The plans, data and events are all within the *Ordering* capability that in turn is within a *Stock managing* capability that is within a *Stock Manager* agent.

²²Prometheus uses the term 'message', whereas JACK uses the term 'event'.

²³JACK does not allow for spaces in names and so we either remove spaces (and use capitals to indicate multiple words) or replace spaces with underscores.

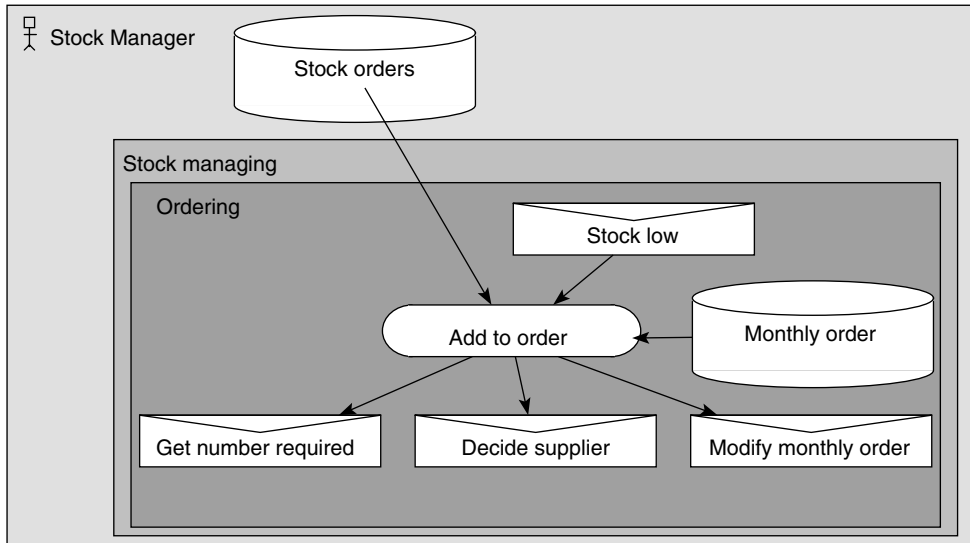


Figure 10.2 Nesting of capabilities within the Stock Manager agent

An agent declaration in JACK takes the form

```
agent AgentName extends Agent {
    ...
}
```

The body ('...' above) consists of declarations specifying what capabilities and plans the agent has, what beliefsets it has and what events it handles, sends and posts.

The *Stock Manager* agent has a database called *Stock Orders* and has *Stock managing* capability. Having a capability is expressed using a declaration of the form

```
#has capability CapabilityName handle;
```

The *Stock Manager* agent has a single beliefset of type *Stock Orders*, which is accessed by the capabilities and their sub-capabilities and plans. Having a beliefset is expressed using a declaration of the form

```
#private data DataType name;
```

Finally, an agent declaration contains declarations of which events the agent posts, sends and handles. The difference between posting and sending an event is that sending takes place between agents, whereas posting is internal to an agent. JACK treats messages as a type of event and thus there is no distinction between receiving a message (from another agent) and handling an (internally posted) event: both are declared using `handles event`.

We thus have the following agent declaration, which is based on Figure 10.3 (which is a copy of Figure 8.2 on page 103).

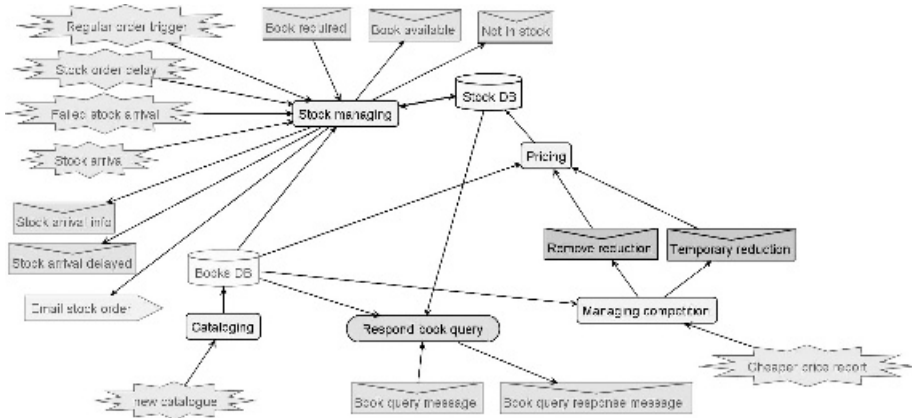


Figure 10.3 Agent overview diagram: stock manager

```

agent StockManager extends Agent {
  #has capability StockManaging sm;
  ...

  #private data Stock_Orders stockorders();
  ...

  /* Percepts */
  #handles external event Stock_order_delay;
  #handles external event Failed_stock_arrival;
  #handles external event Stock_arrival;
  #handles external event Cheaper_price_report;
  #handles external event New_Catalogue;
  ...

  /* Incoming messages */
  #handles external event Log_Outgoing_Books_Message;
  ...

  /* Outgoing message */
  #sends event Stock_arrival_info stockarrivalinfo;
  #sends event Stock_arrival_delayed stockarrivaldelayed;
  ...
}

```

10.3.2 CAPABILITIES

Capability declarations are almost identical to agent declarations. They take the form

```

capability CapabilityName extends Capability {
  ...
}

```

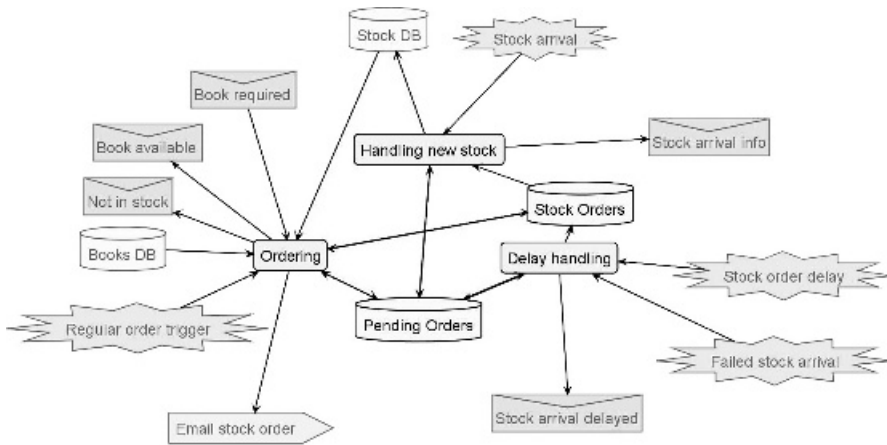


Figure 10.4 Capability diagram: stock managing capability

The body of the declaration is identical in format to that of agent declarations. In this example, the *Stock Managing* capability has *Ordering* as a sub-capability. Both capabilities access the agent's *Stock Orders* beliefset, which is declared using `import data` rather than `private data`. This indicates that the database is external to the capability. In addition to accessing the agent's *Stock Orders* database, the *Ordering* capability also has a private database of *Monthly Order*.

The following JACK code is based on Figures 10.4 and 10.5 (which are copies of Figures 9.1 and 9.2 on page 111).

```
capability Stockmanaging extends Capability {
    #has capability Ordering ordercap;
    ...

    /* import agent's databases */
    #imports data Stock_Orders stockorders();
    ...

    /* Percepts */
    #handles external event Stock_order_delay;
    #handles external event Failed_stock_arrival;
    #handles external event Stock_arrival;
    ...

    /* Incoming messages */
    #handles external event Book_required;
    ...

    /* Outgoing messages (external to agent) */
    #sends event Stock_arrival_info stockarrivalinfo;
    #sends event Stock_arrival_delayed stockarrivaldelayed;
    ...
}
```

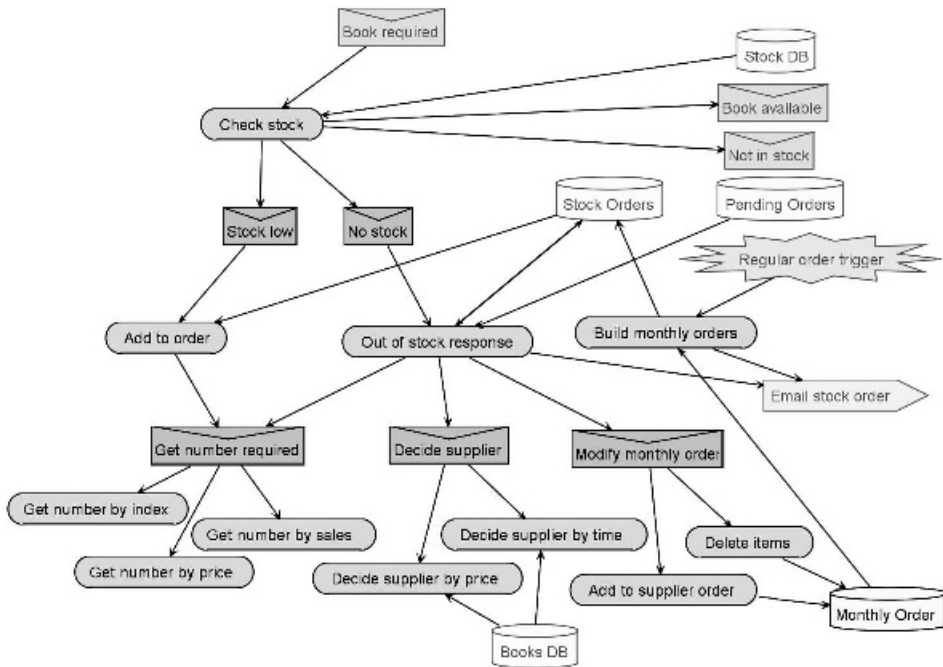


Figure 10.5 Capability overview diagram: ordering capability

```

/* Outgoing events (internal to agent) */
#posts event Book_available;
#posts event Not_in_stock;
...
}

```

The following

```

capability Ordering extends Capability {
  /* import agent's databases */
  #imports data Stock_Orders stockorders();
  #private data Monthly_Order monthlyorder();
  ...

  /* Incoming messages */
  #handles external event Book_Required;
  ...

  /* Internal events */
  #handles event Stock_low;
  #posts event Stock_low;
  #handles event Get_number_required;
  #posts event Get_number_required;
  #handles event Decide_supplier;

```

```

#posts event Decide_supplier;
#handles event Modify_monthly_order;
#posts event Modify_monthly_order;
...

/* Plans that the capability has */
#uses plan Check_stock;
#uses plan Add_to_order;
#uses plan Get_number_by_index;
#uses plan Get_number_by_sales;
#uses plan Get_number_by_price;
...
}

```

10.3.3 DATA

Data can be represented in a range of ways. Data that is represented as objects can, in JACK, be simply programmed in Java.

Data that is represented as relational databases is described using the following descriptor form²⁴:

Data: *name*

Description: Description of the data

Included fields/aspects: What fields does the database contain?

Key(s): Which of these fields are the key, that is, which fields are sufficient to uniquely determine a tuple.

Electronic Bookstore: Case study

Data: Stock_Orders

Description: Stores the amount of each book that is on order. The bookid is the key.

Data type: relational database

Included fields/aspects: bookid (number), amount on order (number)

Persistent: no

External to system: no

Data that is represented as relational databases can be mapped into JACK beliefsets. The *Stock Orders* database maps directly to the following JACK code.

```

beliefset Stock_Orders extends ClosedWorld {
  #key field int bookid
  #value field int amount_on_order
  #indexed query getord(int bookid, logical int amount_on_order);
}

```

²⁴The descriptors in this chapter include information that can be found only in the descriptors. For example, in the data descriptor, the fields 'produced by' can be obtained from the diagrams and so are not included.

The `ClosedWorld` means that the database stores facts, and anything not stored is assumed to be false. An `OpenWorld` database stores both positive and negative information, and anything not stored is assumed to be unknown. Usually, `ClosedWorld` is appropriate.

The declaration of the query uses logical variables to indicate outputs. So, the declaration above automatically generates a method `getord` that takes a `bookid` and looks up the corresponding amount on order.

10.3.4 MESSAGES/EVENTS

Messages are described using the following descriptor form:

Message: *name*

Description: Description of the message

Purpose: What is the aim of the event?

Carried Information: What data does the event carry/contain?

Coverage & Overlap: Will there always be at least one applicable plan?
Will there always be at most one applicable plan instance?

Electronic Bookstore: Case study

Message: *Stock_low*

Purpose: Cause more stock to be ordered

Carried Information: Book record, current stock held (number)

Coverage & Overlap: Covered, no overlap

Message: *Get number required*

Purpose: Determine how many copies to order

Carried Information: Book record, current stock held, number

Coverage & Overlap: not covered: There can be situations where no plan matches, if for instance not all books have a category field, overlap: *Get number by sales*, *Get number by price* and *Get number by index* can all overlap. Preference ordering is sales, price, index.

Message: *Decide supplier*

Purpose: Decide on which supplier to use

Carried Information: Book record, current stock held, amount to order

Coverage & Overlap: Covered, no overlap

Message: *Modify monthly order*

Purpose: Modify the monthly order

Carried Information: Book record, current stock held, amount to order, supplier to use

Coverage & Overlap: Covered, no overlap

Mapping these descriptors to a JACK implementation is straightforward. The JACK code for each event declares the data carried by the event and provides a posting method that creates the event.

One issue that needs to be considered in JACK is the *type* of the event: JACK provides a number of event types with different properties including `MessageEvent`

(for messages between agents) and `BDIGoalEvent` for goals. As a rough rule, an event should be of type `BDIGoalEvent` when there are multiple plans that can handle the event and it should be of type `MessageEvent` when it is being sent between agents.

The message `get_number_required` is not covered. This means that an attempt to determine how many copies to order could fail. This could be addressed either by requiring that any plan that posts `get_number_required` be prepared to fail, or by adding additional plans to handle the message. For example, one could add a plan that simply asked a clerk how many books to order. Finally, another alternative is to make (and document!) assumptions that will guarantee that the message is covered, for example, assuming that all books have categories will ensure that the message is covered.

```
event Stock_low extends BDIGoalEvent {
    BookRecord bookrecord;
    int currentstock;

    #posted as stocklow(BookRecord br, int st) {
        currentstock = st;
        bookrecord = br;
    }
}

event Get_number_required extends BDIGoalEvent {
    BookRecord bookrecord;
    int currentstock;
    int amount;

    #posted as get_number_required(BookRecord br, int st) {
        currentstock = st;
        bookrecord = br;
    }
}

event Decide_supplier extends Event {
    BookRecord bookrecord;
    int currentstock;
    int amount;
    String supplier;

    #posted as decide_supplier(BookRecord br, int st, int am) {
        currentstock = st;
        bookrecord = br;
        amount = am;
    }
}

event Modify_monthly_order extends Event {
    BookRecord bookrecord;
    int currentstock;
    int amount;
    String supplier;
```

```

#posted as modify_monthly_order(
    BookRecord br, int stock, int am, String supp)
{
    currentstock = stock;
    bookrecord = br;
    amount = am;
}
}

```

10.3.5 PLANS

Finally, each plan in the Prometheus detailed design maps to a JACK plan. Plans are described using the following descriptor form:

Plan: *name*

Description: Description of the plan

Trigger: What message or percept causes the plan to execute?

Context: In what situation should this plan be applicable?

Data used & produced: What data does this plan read and/or write?

Goal: What is the goal achieved by this plan?

Failure: When might the plan fail?

Failure recovery: What needs to be done to recover from failure?

Procedure: What is the sequence of steps that the plan performs?

Electronic Bookstore: Case study

Plan: *Add to order*

Description: Order more stock of a book.

Trigger: *Stock low*

Context: Book not already ordered

Data used & produced: Reads *Stock Orders* and *Monthly Order*

Goal: Have books in stock

Failure: never fails

Failure recovery: not applicable

Procedure:

1. Determine how much stock to order (*Get number required*)
2. Determine which supplier to use (*Decide supplier*)
3. Add the order to the monthly order (*Modify monthly order*)

This plan can be implemented in JACK with the code below. The important point is that the mapping is one-to-one: each plan in the detailed design becomes a single-plan declaration in a BDI platform such as JACK. The mechanism for running plans in response to events being posted is provided by the BDI platform.

The information in the plan descriptor also gives much of the information in the JACK plan declaration. For example, the `handles` event declaration is the trigger in the plan descriptor and the context condition is given in pseudo-code in the descriptor.

The JACK plan declaration needs to declare the events posted and sent by the plan as well as data access. This information can be seen in the capability overview diagram where the plan appears. Finally, the plan body needs to be expressed in code rather than pseudo-code. In JACK, the language used to write plan bodies is Java extended with '@statements' such as `@achieve` or `@send`.

The plan body below follows the four steps in the plan descriptor. Steps two, three and four are realized by posting an event as a sub-task of the current plan using `@subtask`. In all three cases, the processing of the plan suspends until the event either succeeds or fails. The difference between `@subtask` and `@post` is that the latter is asynchronous – it does not suspend the plan while the event is handled.

 **TIP:** Often, when a plan posts an event to trigger other plans, there will be some information that results from the sub-tasks. For example, posting *Decide supplier* results in a supplier being chosen. One way of communicating this information back to the parent plan is to have an additional field in the event that is set by the child plan.

```
plan Add_to_order extends Plan {
  /* Trigger */
  #handles event Stock_low stocklow;

  /* Events/messages sent */
  #posts event Get_number_required getnumber;
  #posts event Decide_supplier decidesupplier;
  #posts event Modify_monthly_order modifymonthlyorder;

  /* Data read/modified */
  #reads data Stock_Orders stockorders;
  #reads data Monthly_Order monthlyorder;

  /* Context condition */
  context() {
    stockorders.getord(stocklow.bookrecord.bookid,sorder)
      && sorder.as_int()==0 && ... ;
  }

  /* The body of the plan */
  body() {
    logical int sorder;
    /* Create event */
    getnumber = getnumber.get_number_required(
      stocklow.bookrecord, stocklow.currentstock);
    /* Post the event */
    @subtask(getnumber);
```

```

decidesupplier = decidesupplier.decide_supplier(stocklow.bookrecord,
        stocklow.currentstock, getnumber.amount);
@subtask(decidesupplier);

modifymonthlyorder = modifymonthlyorder.modify_monthly_order(
        stocklow.bookrecord, stocklow.currentstock,
        getnumber.amount, decidesupplier.supplier);
@subtask(modifymonthlyorder);
} // body()
} // plan Add_to_order

```

10.4 AUTOMATIC GENERATION OF SKELETON CODE

In this chapter, we have shown how the results of a Prometheus detailed design map naturally to (skeleton) code in an agent-oriented programming language, specifically JACK. In fact, much of this code can be automatically generated from a structured design. The JACK Intelligent Agents environment contains a tool, ‘JACK Development Environment’ (JDE), which supports this automatic generation of skeleton (JACK) code from a graphical design.

We have developed a prototype tool ‘Prometheus Design Tool’ (PDT), available at www.cs.rmit.edu.au/agents/pdt. This tool supports the process described in this book, of system specification, architectural design and detailed design. The detailed design produced by PDT can be straightforwardly converted to JDE, and consequently to JACK code. A similar approach could be used to develop plug-ins that enable PDT to produce skeleton code for a range of agent-programming systems. It is our hope that the structured methodology presented, combined with tools that support the process, will assist developers in exploring the powerful agent-programming paradigm.